

Ultimate Amiga OS

Technical Reference Manual

Version 0.1.0-dev

May 31, 2026

UAOS is a standalone, bare-metal 64-bit operating system for x86_64 platforms. It is derived from the AROS open-source Amiga replacement kernel and features transparent M68k JIT emulation and native GNU tool integration.

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Architectural Constraints	4
2	System Architecture	5
2.1	Boot Sequence	5
2.2	M68k Thunk Dispatch	5
2.3	MMU Sandbox	6
3	Kernel Subsystems	7
3.1	ROM Module Registry (<code>rom_modules.c</code>)	7
3.2	Thunk Handler (<code>thunk_handler.c</code>)	7
3.3	Page Fault Handler (<code>page_fault_handler.c</code>)	7
3.4	Virtual Filesystem Layer (<code>vfs.c</code> , <code>ramfs.c</code>)	7
3.5	Block Device Layer (<code>blockdev.c</code>)	8
4	Build System	9
4.1	Prerequisites	9
4.2	Building the ISO	9
4.3	Testing in QEMU	9
5	Graphics and Window Manager	11
5.1	Framebuffer Initialisation	11
5.2	Desktop	11
5.3	Window Manager	11
5.3.1	Data Structures	11
5.3.2	Z-Order and Focus	12
5.3.3	Drag and Resize	12
5.4	Software Cursor	12
6	Shell and Built-in Commands	13
6.1	Built-in Commands	13
6.2	Filesystem Operations	13
7	Troubleshooting	15
7.1	GRUB: “Invalid arch-dependent ELF magic”	15
7.2	GRUB: “You need to load the kernel first”	15
7.3	Kernel halts immediately after boot	15
A	Amiga Memory Map Reference	16

List of Figures

List of Tables

2.1	UAOS Guest Address Space Map	6
6.1	UAOS Shell Built-in Commands	14

Chapter 1

Introduction

1.1 Project Overview

Ultimate Amiga OS (UAOS) is a standalone bare-metal x86_64 operating system designed to provide a native Amiga-compatible environment on modern hardware without requiring any host operating system. It achieves this through:

- An extended AROS microkernel providing Exec, DOS, and Intuition semantics on 64-bit hardware.
- A transparent M68k JIT emulation sandbox that executes legacy Amiga “Hunk” binaries inside a 4 GB virtual address space.
- An illegal-opcode trap mechanism (`ILLEGAL/0x4AFC`) that intercepts system calls at the ROM vector level and dispatches them to native 64-bit C implementations.
- An MMU sandbox using 2 MB huge pages with hardware-register windows deliberately left non-present to transparently virtualise custom chipset I/O via `#PF` exceptions.

1.2 Architectural Constraints

1. **No Linux kernel.** UAOS boots directly into its own x86_64 AROS-derived microkernel. No Linux, no POSIX kernel layer.
2. **Trap-based thunking.** M68k Hunk binaries are loaded into the 4 GB sandbox. System calls are intercepted via `ILLEGAL + $414D` magic-token sequences routed to native 64-bit AROS libraries.
3. **MMU hardware virtualisation.** x86_64 page tables map the sandbox with 2 MB huge pages. The Amiga hardware register window `0x00B00000–0x00DFFFFFF` is marked `NOT PRESENT`.
4. **Coexisting filesystem namespace.** `dos.library` handlers resolve Amiga assigns (`SYS:`, `S:`, `C:`, `RAM:`) alongside POSIX translation paths.
5. **AROS Kickstart firmware.** The open-source AROS replacement ROM is used as the modular firmware base with vectors patched at build time.

Chapter 2

System Architecture

2.1 Boot Sequence

1. GRUB2 loads the ELF64 kernel via the Multiboot2 protocol.
2. `uaos_kernel_entry.asm` transitions the CPU from 32-bit protected mode into 64-bit long mode, sets up a bootstrap identity-map page table, and calls `uaos_kernel_main()`.
3. The C kernel entry point validates the Multiboot2 magic value, initialises the VGA text-mode console and UART, then calls:
 - `FB_Init()` — parses the Multiboot2 framebuffer tag.
 - `IDT_Init()` — installs 256-vector IDT and remaps 8259A PIC.
 - `UAOS_MMU_Init()` — installs the sandbox page tables.
 - `UAOS_ROM_RegisterAll()` — populates the ROM module registry.
 - `UAOS_Bridge_Init()` — allocates the 4 GB guest RAM window.
 - `PS2Mouse_Init()` and `PS2Kbd_Init()` — PS/2 input drivers.
 - `RTC_Init()` — CMOS real-time clock with UIE interrupt.
 - `VFS_Init()` — virtual filesystem layer and RAM: mount.
 - `BlockDev_Init()` — block device layer.
 - `virtio_blk_init()` — VirtIO block device detection.
 - `Desktop_Draw()` and `ShellWin_Init()` — graphical shell.
4. Enters the `hlt`-loop event dispatcher.

2.2 M68k Thunk Dispatch

The thunk mechanism replaces selected Exec jump-table vectors in the AROS ROM with 8-byte breakout stubs assembled by the `MK_THUNK` macro in `emulation/rom_patches/rom_traps.s`:

Listing 2.1: `MK_THUNK` macro

```
1 MK_THUNK MACRO
2     ILLEGAL                ; $4AFC --- triggers JIT breakout
3     dc.w    \1             ; UAOS magic token ($414D = "AM")
4     dc.w    \2             ; function index (1-based)
5     rts                   ; return to guest after dispatch
6 ENDM
```

When the JIT core decodes `0x4AFC` it invokes `UAOS_Bridge_IllegalOpcode()`. The C dispatcher:

1. Reads the two words immediately following the `ILLEGAL` opcode.
2. Validates the `$414D` signature.
3. Extracts arguments from the M68k register state (`A1`, `D0`, `D1`, ...).
4. Calls the native 64-bit stub.
5. Writes return values back into `D0/A0`.
6. Advances the guest PC by 6 bytes.

2.3 MMU Sandbox

`kernel/exec/mmu_sandbox.c` constructs four-level `x86_64` paging tables mapping the entire 4 GB guest address space using 2 MB huge pages. Table 2.1 shows the critical address assignments.

Address Range	Size	Description
<code>0x00000000-0x001FFFFFFF</code>	2 MB	Chip RAM
<code>0x00200000-0x009FFFFFFF</code>	8 MB	Fast RAM (lower)
<code>0x00B00000-0x00DFFFFFFF</code>	3 MB	Custom chip registers (NOT PRESENT)
<code>0x00F80000-0x00FFFFFFF</code>	512 KB	Kickstart ROM mirror
<code>0x01000000-0x7FFFFFFF</code>	2 GB	Extended Fast RAM
<code>0x80000000-0xFFFFFFFF</code>	2 GB	<code>x86_64</code> kernel space

Table 2.1: UAOS Guest Address Space Map

Pages in the hardware register window have `PAGE_PRESENT` cleared and `PAGE_NO_CACHE` set. Any access faults to this region are caught by the `#PF` handler in `kernel/exec/page_fault_handler.c` and forwarded to the chip emulator.

Chapter 3

Kernel Subsystems

3.1 ROM Module Registry (`rom_modules.c`)

`UAOS_ROM_RegisterAll()` is called once at boot. It populates a static array of `UaosRomModule` descriptors, each holding:

- Library name string (e.g. `"exec.library"`).
- Library version number.
- 32-bit Amiga base address.
- Array of native function pointers (indexed 1-based to match `rom_traps.s` function indices).

3.2 Thunk Handler (`thunk_handler.c`)

The `UAOS_AMIGA_TO_HOST()` macro translates any 32-bit Amiga guest address to the corresponding host linear address:

Listing 3.1: Address translation macro

```
1 #define UAOS_AMIGA_TO_HOST(amiga_addr) \
2   ((void *)((uintptr_t)(uaos_ram_base) + (uint32_t)(amiga_addr)))
```

3.3 Page Fault Handler (`page_fault_handler.c`)

The ISR entry stub `uaos_page_fault_isr` (written in inline GAS assembly) saves all 15 GPRs onto the kernel stack, computes pointers to the `InterruptFrame` and `SavedRegs` blocks, and calls the C function `UAOS_PageFaultHandler()`. On return from the C handler, GPRs are restored and `IRETQ` resumes execution past the faulting instruction.

3.4 Virtual Filesystem Layer (`vfs.c`, `ramfs.c`)

UAOS provides a thin VFS dispatch layer over an in-memory RAM filesystem. Paths use AmigaDOS format: `VOL:path/to/file`.

Listing 3.2: VFS file handle

```
1 typedef struct {
2     RamFsNode *node;    /* NULL = invalid / not open */
3     uint32_t   pos;     /* current read/write position */
4 } VfsFile;
```


RAM Filesystem (`ramfs.c`): BSS-backed storage with 256 nodes and a 64 KB data pool. Auto-mounted at boot as `RAM:` with `T`, `ENV`, `CLIPS`, and `S` directories.

Partition volumes: The block device layer detects MBR partitions on VirtIO devices and registers them as child block devices (e.g. `virtio01`). Formatted partitions are auto-mounted in the VFS by their display name (e.g. `DH0:`) and by their FAT32 volume label (e.g. `WORK:`), both backed by RAMFS volumes.

VFS Operations include `VFS_Open`, `VFS_Read`, `VFS_Write`, `VFS_Mkdir`, `VFS_Delete`, `VFS_ResolveDir`, `VFS_MountPartition`, `VFS_GetMountCount`, and `VFS_GetMountName`.

3.5 Block Device Layer (`blockdev.c`)

Unified interface for storage devices. Key operations:

- `BlockDev_Register(dev)` / `BlockDev_Unregister(dev)`
- `BlockDev_RegisterPartition(parent, idx, start, count, name)` — creates child partition devices from MBR entries.
- `BlockDev_CheckFormatted(dev)` — reads boot sector for valid FAT BPB signature.
- `BlockDev_ReadVolLabel(dev, buf, max)` — extracts the 11-byte FAT32 volume label from boot sector offset 71.
- `BlockDev_Read` / `BlockDev_Write` / `BlockDev_GetCapacity`

VirtIO Block Driver (`virtio_blk.c`): PCI scanning for vendor `0x1AF4` / device `0x1001`, capacity reporting from configuration space. Virtqueue I/O is pending implementation.

Chapter 4

Build System

4.1 Prerequisites

Listing 4.1: Install build dependencies (Ubuntu/Debian)

```
1 sudo apt install nasm gcc binutils xorriso \  
2     grub-pc-bin grub-common \  
3     binutils-m68k-linux-gnu python3
```

4.2 Building the ISO

Listing 4.2: Full build pipeline

```
1 # Assemble NASM kernel entry  
2 nasm -f elf64 kernel/boot/uaos_kernel_entry.asm \  
3     -o build/uaos_kernel_entry.o  
4  
5 # Compile C kernel sources (freestanding)  
6 gcc -ffreestanding -fno-stack-protector -fno-pie \  
7     -mno-red-zone -nostdlib -m64 -O2 \  
8     -c kernel/boot/uaos_kernel_main.c -o build/uaos_kernel_main.o  
9 # ... (see scripts/build_iso.sh for full compile step list)  
10  
11 # Link ELF64 kernel  
12 ld -T kernel/boot/uaos_kernel.ld build/*.o -o build/uaos-kernel.elf  
13  
14 # Package ISO  
15 ./scripts/build_iso.sh
```

4.3 Testing in QEMU

Copy the OVMF variables file once (required for UEFI variable store):

Listing 4.3: One-time OVMF setup

```
1 cp /usr/share/OVMF/OVMF_VARS_4M.fd /tmp/ovmf_vars.fd
```

Listing 4.4: QEMU test invocation

```
1 qemu-system-x86_64 \  
2     -machine q35,usb=off \  
3
```

```
3 -drive if=pflash,format=raw,readonly=on,\  
4 file=/usr/share/OVMF/OVMF_CODE_4M.fd \  
5 -drive if=pflash,format=raw,file=/tmp/ovmf_vars.fd \  
6 -cdrom build/Ultimate_Amiga_OS.iso \  
7 -m 512M -vga virtio -no-reboot
```

The `usb=off` flag is required to prevent QEMU's USB tablet device from conflicting with the PS/2 relative mouse driver. Add `-serial stdio` to see kernel debug output on the terminal.

Chapter 5

Graphics and Window Manager

5.1 Framebuffer Initialisation

GRUB2 fills the Multiboot2 framebuffer tag (type 8) from the UEFI GOP driver. `FB_Init()` parses this tag to populate the global `FbState g_fb` structure, which holds the physical base address, pixel dimensions, pitch (bytes per row), and bit depth (24 or 32 bpp).

All drawing primitives (`FB_FillRect`, `FB_DrawHLine`, `FB_DrawVLine`, `FB_PutPixel`, `FB_PutStr`) clip coordinates to the framebuffer bounds before writing, supporting windows that extend partially off-screen.

5.2 Desktop

`Desktop_Draw()` renders the full Workbench-style backdrop:

- **Menu bar** (top, 20 px) — Workbench menu labels and clock area.
- **Backdrop** — light grey base with a dark-grey stipple dot overlay, one dot per two pixels alternating rows/columns (classic Amiga pattern).
- **Disk icons** — top-right corner, stacked vertically.
- **Information window** — centred boot status window.
- **Status bar** (bottom, 18 px) — kernel idle message and RAM size.

`Desktop_RedrawRect(rx, ry, rw, rh)` repaints a sub-rectangle of the desktop (backdrop + overlays) without a full-screen repaint. This is used by the window manager to erase the old window footprint during drag and resize, avoiding the full-screen flicker that would otherwise occur.

5.3 Window Manager

`kernel/display/wm.c` implements a lightweight window manager supporting up to eight windows simultaneously.

5.3.1 Data Structures

Listing 5.1: WmWindow structure

```
1 typedef struct {  
2     int     x, y, w, h;
```

```
3     char        title[32];
4     WM_DrawFn   draw;        /* repaint callback: (x, y, w, h) */
5     WM_KeyFn    on_key;      /* keystroke callback: (char c) */
6     int         active;
7 } WmWindow;
```

5.3.2 Z-Order and Focus

Windows are stored in `g_wins[]` and indexed by a separate `g_zorder[]` array (`[0]` = back, `[g_nwins-1]` = top). `WM_Redraw()` paints windows back-to-front. Clicking a window calls `raise_window()` to promote it to the top of the z-stack.

5.3.3 Drag and Resize

- **Drag** — a mouse-down on the title bar records the grab offset. Each subsequent mouse move erases the old footprint with `Desktop_RedrawRect`, updates `w->x/y`, and repaints all windows.
- **Resize** — a mouse-down on the 16×16 resize grip in the bottom-right corner records the base size and origin. Mouse movement computes the new dimensions relative to the drag start point.
- Windows may extend off the left, right, and bottom screen edges. The title bar is always constrained to remain on-screen.

5.4 Software Cursor

`kernel/display/cursor.c` implements a 16×16 Amiga-style arrow sprite cursor. The cursor is rendered by saving the background pixels beneath it, drawing the sprite, and on each move restoring the saved background before re-saving and re-drawing at the new position. This ensures the desktop and window content are never permanently overwritten by cursor movement.

Chapter 6

Shell and Built-in Commands

The shell window (`shell_win.c`) registers with the window manager via `WM_AddWindow()` and implements a scrollable terminal with up to 128 lines of history and an input line with backspace support.

6.1 Built-in Commands

6.2 Filesystem Operations

The shell integrates with the VFS layer for all filesystem operations. Partition volumes (e.g. `DH0:`, `WORK:`) can be accessed with `cd` and `dir` once auto-mounted at boot or after formatting with the `format` command.

Command	Description
help	List available commands
version	Show OS version and architecture
mem	Display memory information
clear	Clear shell history
reboot	Warm reboot
libs	List loaded kernel libraries
dir [path]	List directory contents
cd [path]	Change or show current directory
mkdir <path>	Create a directory
delete <path>	Delete a file or empty directory
type <file>	Display file contents
copy <src> <dst>	Copy a file
rename <from> <to>	Rename or move a file
pwd	Print working directory
echo <text>	Print text
pointer	Open pointer preferences
protect <flags> <path>	Set attributes ($\pm r$, $\pm h$)
attr <path>	Show file attributes
info [device]	Show mounted disks and volumes
alias [name cmd]	Create or list aliases
unalias <name>	Remove an alias
set [name val]	Set or list environment variables
unset <name>	Remove a variable
date	Show date and time
which <cmd>	Locate a command
disks	List block devices
fdisk <device>	Partition a block device
format <dev> [fs]	Format a partition (FAT32)
run <prog> [args]	Run an embedded Amiga binary

Table 6.1: UAOS Shell Built-in Commands

Chapter 7

Troubleshooting

7.1 GRUB: “Invalid arch-dependent ELF magic”

This error means GRUB received a file that is not a valid ELF64 executable. Ensure `uaos-kernel.elf` was produced by the full NASM + GCC + LD pipeline (not the Python mock generator used for structural validation only).

7.2 GRUB: “You need to load the kernel first”

GRUB could not find the `multiboot2` header within the first 32 KB of the binary. Verify the linker script places `.multiboot2` first and that the NASM source emits the section correctly.

7.3 Kernel halts immediately after boot

Enable serial output and check the UART log:

```
1 qemu-system-x86_64 -cdrom build/Ultimate_Amiga_OS.iso \  
2 -m 512M -boot d -serial stdio 2>&1 | tee boot.log
```


Appendix A

Amiga Memory Map Reference

See Table [2.1](#) in Chapter 2. The full OCS/ECS/AGA custom chip register map follows Commodore's *Hardware Reference Manual, 3rd Ed.*